ARI-SPEC

ARI — Agent Runtime Interface

Version: 0.1 (Draft) **Status:** Draft — Request for Comments
Reference implementation: `marrow` **License:** Apache-2.0 (spec text under CC BY 4.0 — see §13)

Abstract

The **Agent Runtime Interface (ARI)** is a language-neutral contract for the *runtime layer* of an LLM agent system: the loop that holds conversation state, calls a model provider, dispatches tools, routes messages between agents, and manages cancellation, timeouts, persistence, and observability.

ARI does **not** standardize *what* tools exist, *how* a model is prompted, or *how* agents talk to each other across a network — those layers are owned by other specifications (notably **MCP** and **A2A**; see §1.3). ARI standardizes the layer *beneath* them: the embeddable engine that actually runs an agent turn.

A runtime that implements this contract is **ARI-conformant** and can be substituted for any other ARI-conformant runtime without changing application code that targets the interface. `marrow` is the reference implementation.

---

1. Introduction

1.1 Motivation

Today's agent frameworks (LangGraph, CrewAI, AutoGen) are excellent and mature, but they share an architectural assumption: **the runtime is written in Python and runs on a server with a network path to a hosted model.** That assumption fails in a growing class of deployments:

- **Embodied / robotic** systems with hard real-time constraints, where garbage-collection pauses and a heavyweight interpreter on the hot path are unacceptable.
- **Edge / on-device** deployments on resource-constrained hardware, often with a local model and no reliable network.
- **Native applications** (game engines, audio/DSP, trading systems, CAD) that need to embed an agent loop without shipping a Python runtime to the customer.

These deployments cannot adopt a Python-locked runtime. Yet there is no neutral contract that says *what an agent runtime must expose* — so every native re-implementation reinvents the surface, and nothing is interoperable.

ARI fills that gap. It defines the contract once, so that:

- A native runtime (C++, Rust, Zig, …) can claim conformance and be a drop-in.
- Tooling (tracers, state stores, provider adapters) can target the contract rather than a specific framework.
- "Agents you can't ship Python into" becomes a portable, standardized target.

1.2 Scope

ARI standardizes:

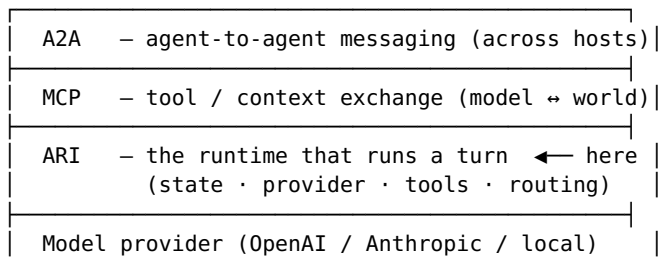
Area	Section
Message / role data model	§2
Model provider interface (generate, stream, cancel, timeout)	§3
Agent conversation state	§4
Tool registration and invocation contract	§5
Multi-agent routing (send, drain, handoff, backpressure)	§6
Runtime lifecycle (shutdown)	§7
State persistence	§8
Observability (trace spans)	§9
Conformance profiles	§10

ARI explicitly does NOT standardize:

- The wire protocol for tool *definitions* or context exchange — that is **MCP**.
- Cross-process / cross-network agent-to-agent messaging — that is **A2A**.
- Prompt formats, model weights, or inference engines.
- Any specific programming language's binding ergonomics (those are implementation choices; see Appendix A).

1.3 Relationship to MCP and A2A

ARI is **complementary, not competitive**, with the emerging agent protocol stack:



└──┘

An ARI runtime SHOULD be able to expose its tools over MCP and participate in an A2A mesh, but neither is required for ARI conformance. ARI is the layer you embed; MCP and A2A are protocols you speak.

1.4 Terminology

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

- **Runtime** — an implementation of this interface.
- **Agent** — a named, stateful conversation participant within a runtime.
- **Provider** — an adapter that turns a request into a model completion.
- **Turn / Step** — one provider invocation that appends to agent state.
- **Host application** — the code embedding the runtime.

1.5 Conformance overview

Conformance is claimed against a **profile** (§10). Every conformant runtime MUST satisfy **ARI-Core**. Runtimes targeting embedded or server deployments MAY additionally satisfy **ARI-Embedded** or **ARI-Server**. A runtime MUST NOT advertise "ARI-conformant" without naming the profile(s) it passes and the version of the conformance suite used.

---

2. Data Model

2.1 Role

A message role MUST be one of exactly four values:

Role	Meaning
`system`	Instructions that condition the agent.
`user`	Input from outside the agent.
`assistant`	Output produced by the agent / model.
`tool`	The result of a tool invocation, fed back into context.

Runtimes MUST NOT define additional roles in the core data model. Vendor extensions belong in message metadata (§2.2).

2.2 Message

A **Message** is the atomic unit of conversation state. It MUST carry:

Field	Type	Required	Notes
`role`	Role	yes	See §2.1.
`content`	UTF-8 string	yes	The message body. May be empty.
`name`	string	no	Author/tool name; empty if unset.
`timestamp_ms`	int64	no	Unix epoch milliseconds; `0` if

			unset.
`metadata`	map<string,string>	no	Opaque key/value extension point.

Runtimes MUST treat `metadata` as opaque pass-through and MUST preserve it across persistence and routing. Unknown keys MUST NOT cause errors.

2.3 Content limits

A runtime MUST enforce a maximum `content` size and MUST reject (not truncate) messages exceeding it, surfacing an error to the caller. The default ceiling is **4 MiB**. Runtimes targeting **ARI-Embedded** (§10.2) MAY set a lower fixed ceiling but MUST document it. This protects against out-of-memory via runaway tool output.

---

3. Provider Interface

A **Provider** turns a `GenerationRequest` into a completion. Providers are the ARI extension point for model vendors and local inference engines.

3.1 GenerationRequest

Field	Type	Required	Default	Notes
`model`	string	yes	—	Provider-specific model id.
`messages`	Message[]	yes	—	Ordered conversation context.
`temperature`	float	no	`0.7`	Sampling temperature.
`max_tokens`	int	no	`1024`	Upper bound on completion length.
`timeout_ms`	int	no	`0`	Wall-clock timeout; `0` disables.
`cancel_token`	CancelToken	no	none	Cooperative cancellation handle.

3.2 GenerationResponse

Field	Type	Required	Notes
`content`	string	yes	The completion text.
`prompt_tokens`	int	no	`0` if the provider does not report.
`completion_tokens`	int	no	`0` if the provider does not report.

Token counts are best-effort. A provider that cannot report usage MUST return `0`, never a fabricated estimate.

3.3 `generate` (synchronous)

A provider MUST implement a synchronous `generate(request) -> response`. It MUST honor `cancel_token` at natural yield points and SHOULD honor `timeout_ms`; if it does not honor the timeout internally, the runtime MUST enforce it externally (e.g. by running the call on a worker and abandoning it). On cancellation or timeout the provider/runtime MUST surface a distinguishable error, not a partial success.

3.4 `generate_stream` (streaming)

A provider SHOULD implement `generate_stream(request, on_chunk)`, invoking the callback once per chunk such that concatenating all chunks reconstructs the full `content`. A provider that cannot stream MUST fall back to emitting the entire `content` as a single chunk so that callers written against the streaming API still work. Conformance to streaming is part of **ARI-Server**; it is OPTIONAL in **ARI-Core**.

3.5 Cancellation

A runtime MUST provide a **CancelToken** primitive supporting:

- `cancel()` — request cancellation (idempotent).
- `cancelled()` — non-blocking check.
- `reset()` — clear state for reuse.

Cancellation is cooperative: long-running providers and tools MUST poll `cancelled()` at yield points. CancelToken operations MUST be safe to call from a thread other than the one running the request.

3.6 Timeouts

`timeout_ms` is wall-clock. A value of `0` disables the timeout. When a timeout elapses the runtime MUST stop waiting and surface a timeout error; it SHOULD also trip the request's `cancel_token` so cooperating providers can release resources.

---

4. Agent State

An **AgentState** holds the ordered message history and optional system prompt for one agent. It MUST expose:

Operation	Contract
`id()`	Stable, immutable agent identifier.
`append(message)`	Append to history; MUST be safe under concurrent readers.
`history()`	Snapshot of all messages in order.
`size()`	Current message count.
`clear()`	Drop all messages (system prompt retained unless re-set).
`set_system_prompt(text)` / `system_prompt()`	Set/get the conditioning prompt.
`trimmed(n)`	Return at most the most-recent `n` messages, in order.

State reads MUST be consistent: a `history()` snapshot MUST NOT observe a partial `append`. Implementations are RECOMMENDED to use read-heavy synchronization (e.g. a shared/reader-writer lock) since history is read far more than written.

`trimmed(n)` MUST preserve chronological order and MUST NOT drop a `system` message that the runtime injects separately; trimming applies to conversational history, and the system prompt is prepended at request-build time.

---

5. Tool Interface

5.1 Tool descriptor

A tool MUST be registered with:

Field	Type	Required	Notes
`name`	string	yes	Unique within a registry.
`description`	string	no	Human/model-facing summary.
`schema_json`	string	no	JSON Schema of the arguments.
`fn`	callable	yes	The implementation (see §5.2).

The `schema_json` field is a **string** containing JSON Schema, stored verbatim. ARI does not mandate how the schema is produced (hand-written, generated from type annotations, etc.).

5.2 Invocation contract

The tool implementation is a pure **JSON-in / JSON-out** function:

```
invoke(name: string, args_json: string) -> result_json: string
```

- `args_json` MUST be a JSON object string.
- The runtime MUST raise a distinguishable error if `name` is not registered.
- The implementation language is irrelevant to the contract: the reference implementation keeps the registry and dispatch in C++ while tool bodies may be written in the host language.

5.3 Result envelope

A tool invocation result MUST be a JSON object using this envelope:

```
{ "ok": true,  "result": <any JSON value> }
{ "ok": false, "error": "<message>" }
```

On failure, the runtime MUST set `"ok": false` and MUST place a human-readable string in `"error"`. The runtime MUST NOT leak internal stack traces or host exception details into `"error"` (see §5.4).

5.4 Limits and redaction

A runtime MUST support configurable size caps on both `args_json` and the result, and MUST redact internal error detail by default. These limits protect the context window and prevent information disclosure. **ARI-Embedded** runtimes MUST enforce a fixed, documented cap.

---

6. Multi-Agent Routing

A runtime MUST provide a **Router** mediating messages between named agents.

6.1 send / drain

Operation	Contract
`register_agent(id)` / `unregister_agent(id)`	Manage routable identities.
`send(from, to, message)`	Enqueue a message into `to`'s inbox.
`drain(id)`	Atomically remove and return all pending messages for `id`.

`drain` MUST be atomic with respect to concurrent `send` calls: a drained message MUST NOT also remain in the inbox, and a message MUST NOT be lost between two drains.

6.2 handoff and active agent

A runtime MUST support `handoff(from, to, seed?)` — transfer of control with an optional seed message — and MUST track an OPTIONAL "active" agent (`active()` / `set_active(id)`). Handoff to an unregistered agent MUST fail without mutating state.

6.3 Backpressure

A runtime MUST allow a per-agent inbox bound and an overflow policy:

Policy	Behavior on full inbox
`Reject`	Throw/return error — backpressure surfaced to the producer.
`DropOldest`	Evict the oldest pending message, accept the new one.
`DropNewest`	Silently discard the incoming message.

The default bound MAY be unbounded (`0`) for **ARI-Server**, but **ARI-Embedded** runtimes MUST default to a finite bound to guarantee bounded memory.

---

7. Lifecycle

A runtime MUST support graceful shutdown:

- `shutdown()` — signal that no new top-level work should start. Idempotent.
- `is_shutdown()` — non-blocking status check.

After `shutdown()`, operations that take ownership of new resources (e.g. creating an agent) MUST fail. In-flight work in other threads MUST be allowed to complete; `shutdown()` MUST NOT forcibly abort running turns. Forcible termination, if offered, MUST be a separate, explicitly-named operation.

---

8. Persistence

Persistence is OPTIONAL for **ARI-Core** and REQUIRED for **ARI-Server**. A runtime that offers persistence MUST do so behind a **StateStore** contract that supports, by behavior:

Operation	Contract
`save(agent_id, state)`	Durably store a snapshot of the agent's history (and system prompt, if any).
`load(agent_id)`	Return the stored messages, or an explicit **absent** result if none.

`delete(agent_id)`	Remove a stored snapshot.
enumerate (e.g. `keys()`)	List the stored agent ids.

A round-trip MUST be **lossless** for all REQUIRED `Message` fields *and* `metadata` (§2.2). A `load` of an unknown `agent_id` MUST return an explicit "absent" result (e.g. `null`/`None`), not an error. At least one durable backend (e.g. SQLite) SHOULD be provided; an in-memory backend MAY be provided for testing. A runtime MAY additionally offer a convenience that rehydrates a live agent directly from a store; the signature of such a helper is an implementation detail, not part of this contract.

---

9. Observability

Observability is OPTIONAL for **ARI-Core** and RECOMMENDED elsewhere. A runtime that emits traces MUST do so behind a **TraceSink** contract that exposes a `span(name, attributes)` operation returning a span handle supporting:

- `set_status(ok: bool)`
- `record_exception(error)`
- scope entry/exit (a span has a clear begin and end).

Runtimes SHOULD provide a no-op sink (zero overhead when disabled) and SHOULD offer an OpenTelemetry-compatible sink. A disabled TraceSink MUST add no measurable allocation on the hot path — this is a hard requirement for **ARI-Embedded**.

---

10. Conformance Profiles

A runtime claims conformance to one or more **profiles**. Profiles are additive: **ARI-Embedded** and **ARI-Server** both presuppose **ARI-Core**.

10.1 ARI-Core

The mandatory baseline. A runtime is **ARI-Core conformant** if it satisfies:

- §2 Data model (all REQUIRED fields, 4 roles, content cap)
- §3.1–3.3, §3.5–3.6 Provider `generate`, cancellation, timeouts
- §4 Agent state
- §5 Tool registration, invocation, result envelope, limits
- §6 Routing (send/drain/handoff) and backpressure policies
- §7 Graceful shutdown

Streaming, persistence, and observability are OPTIONAL at this level.

10.2 ARI-Embedded

The profile this specification is built to serve: agent runtimes embedded in **robotic, edge, and native** hosts. In addition to ARI-Core, an **ARI-Embedded** runtime MUST:

1. **Be embeddable without a managed-language runtime.** The core MUST compile/run as a native library with no required dependency on a garbage-collected interpreter. (Bindings to such languages MAY exist; they MUST NOT be required.)
2. **Bound its memory.** All queues/inboxes MUST default to a finite bound (§6.3); content and tool I/O caps MUST be fixed and

documented (§2.3, §5.4).
3. **Add no hot-path allocation when observability is disabled** (§9).
4. **Support cooperative cancellation and wall-clock timeouts** as first-class, thread-safe primitives (§3.5–3.6).
5. **Declare its dependency footprint.** The minimal build MUST document every third-party dependency; the target is "near-zero."

> **Why this profile is the differentiator:** a Python-locked framework cannot satisfy requirements 1 and 3 without ceasing to be Python-locked. ARI-Embedded defines a lane the incumbent runtimes structurally cannot enter.

10.3 ARI-Server

The profile for high-scale, server-side orchestration. In addition to ARI-Core, an **ARI-Server** runtime MUST satisfy §3.4 (streaming), §8 (persistence with a durable backend), and §9 (observability with an OTel-compatible sink), and SHOULD support concurrent turns with real parallelism (e.g. releasing interpreter locks during provider I/O).

10.4 Claiming conformance

A runtime claiming ARI conformance MUST:

1. Name the profile(s) it passes (e.g. "ARI-Core + ARI-Embedded").
2. State the spec version (e.g. "ARI 0.1") and conformance-suite version.
3. Publish its results from the **ARI Conformance Kit** (see Appendix B and the strategy document).

The phrase "ARI-compatible" without a named profile and version is meaningless and MUST NOT be used in distribution materials.

---

11. Versioning and Stability

- This spec uses **semantic versioning**. Within a major version, changes MUST be additive (new OPTIONAL fields, new profiles) — never a silent tightening of an existing MUST.
- A field's REQUIRED/OPTIONAL status MUST NOT change within a major version.
- Deprecations MUST be announced one minor version before removal and MUST carry a documented migration path.
- The reference implementation's public API stability is governed by its own `STABILITY.md`; spec stability and implementation stability are versioned independently.

---

12. Security Considerations

- **Untrusted tool output** is the primary injection vector. Runtimes MUST apply content (§2.3) and result (§5.4) caps, and MUST redact internal error detail.
- **Resource exhaustion** via unbounded inboxes is mitigated by mandatory bounds in ARI-Embedded (§6.3).
- **Cancellation/timeout** are security primitives, not just ergonomics: they bound the blast radius of a hostile or hung provider/tool.
- **Persistence** stores conversation content; runtimes MUST document the trust boundary of any StateStore backend.

13. Licensing of this document

The ARI specification text is offered under **CC BY 4.0** so that any party may implement it freely and reproduce the contract. The reference implementation (`marrow`) is licensed separately under Apache-2.0. Implementing ARI requires no license to the `marrow` code.

Appendix A: Reference bindings

ARI is language-neutral; the reference implementation provides two bindings that illustrate the contract. These are **informative**, not normative — a conformant runtime need not match these signatures, only the behavior in §§2–10.

C++ (native core): `Provider`, `AgentState`, `AgentRouter`, `ToolRegistry`, `Engine`, `CancelToken`, `Message`, `GenerationRequest/Response` — see `src/core/engine.hpp`.

Python (host binding): `Agent`, `Runtime`, `tool`/`ToolBox`, `Graph`, `StateStore`, `TraceSink`, `AsyncRuntime` — see `python/marrow/`.

Appendix B: Conformance checklist

A runtime is **ARI-Core conformant** when every box is checked. ARI-Embedded and ARI-Server add their own checklists (maintained alongside the Conformance Kit).

- ☐ Four roles, no more (§2.1)
- ☐ Message carries all REQUIRED fields; metadata preserved through routing + persistence (§2.2)
- ☐ Content cap enforced by rejection, not truncation (§2.3)
- ☐ `generate` honors cancel token; timeout enforced internally or by runtime (§3.3, §3.6)
- ☐ CancelToken is thread-safe and idempotent (§3.5)
- ☐ AgentState reads are consistent under concurrent append (§4)
- ☐ Tool invocation uses the JSON envelope; unknown tool errors distinguishably (§5.2–5.3)
- ☐ Tool arg/result caps configurable; internal errors redacted (§5.4)
- ☐ `drain` is atomic vs concurrent `send` (§6.1)
- ☐ All three overflow policies implemented (§6.3)
- ☐ `shutdown` is idempotent; blocks new work; lets in-flight finish (§7)

ARI 0.1 (Draft). Reference implementation: marrow. Comments and proposed changes: open an issue tagged `ari-spec`.